ESE280 Embedded Microcontroller Systems Design I

Spring 2024, K. Short revised November 16, 2024 3:47 pm

Laboratory 11: High Speed Data Inputs and Memory Buffers

To be performed the week starting November 17th.

Prerequisite Reading

1. Lecture 12: Memory Addressing Modes
2. AVR128DB48 Data Sheet, Section 27: USART - Universal Synchronous and Asynchronous Receiver and Transmitter.
3. REYAX RYLR896 UART Interface 868/915 MHz LoRa Antenna Transceiver Module.
4. REYAX AT Command Guide.

Overview

When data is entered into an embedded system at a rate that exceeds the rate at which it can be immediately processed, that data must be placed in a memory buffer and processed after the data is accumulated. Most often, the data is transmitted in **bursts**. For our current application a message is a burst of characters, since the characters are sent one after another with essentially no time between frames. However, there often is time between bursts that can be used to process the data. Previously, the time between messages was two seconds, but for this laboratory it will be extended to three seconds.

Design Tasks

The designs in this laboratory involve writing programs to configure USART1 to receive data over a LoRaWAN channel implemented using a RYLR896 Module transceiver. These messages will be sent at 115.2 kbaud. USART3 is used to relay the messages from USART1 to your LCD. The messages will be sent to the LCD at 9600 baud. You will verify the results of your work using the Saleae logic analyzer. Accordingly, characters are coming in to your AVR128DB48 at a rate that is 12 times the rate that they are going out. This is the kind of situation you might have if the baud rates of the LoRaWAN module and the LCD were fixed and not the same.

Design Task 1: Receiving RYLR896 Output at 115.2 kbaud and Relaying it to Your LCD at 9600 baud

This task is similar to Task 3 of Laboratory 09, except the data rate of the asynchronous serial data at the RX input of USART1 is 115.2 kbaud. The data transmitted from USART3 to the LCD remains at the rate of 9600 baud.

You can modify your program `relay_to_LCD` from Laboratory 09 Task 3 to receive the incoming data at the higher data rate. Name the modified program `relay_to_LCD_hs`. Given the

faster baud rate of the incoming data and the fact that the frames that make up RYLR896 message are sent one immediately after the other by the base system, frames will be coming in at a rate 12 times faster than they can be sent out. With only the single change in the program from what it was in Laboratory 09 we can expect that your program will fail and the LCD will not be able to display the correct messages.

Submit your list file for this program as part of your prelab.

Design Task 2: Buffering then Processing the Incoming Data

When an embedded system is receiving a sequence of data bytes and must process that data, there are two alternatives. If there is time enough between each byte being received it might be possible to process each byte of data as soon as it arrives. If there is not time to process the data between receiving each byte, or if the nature of the processing requires all of the bytes to be available at once, then the data must be stored until it is all received. Once all the data needed is received, it can be processed.

In this task and in Task 1 the processing to be done is to write each character received to USART3's TXDATA register. This is what was attempted in Task 1 and failed because of the significant difference in the incoming frame rate and the outgoing frame rate. The approach in this task is to store the data in a buffer in SRAM until all the characters have been received. Once all the characters are in the SRAM buffer, they can be written to USART3's TXDATA register one at a time at the highest rate compatible with the baud rate of USART3.

Draw a **flowchart** for your program. Write the **code** for your program and verify and debug it. Name the buffer `buffer` and make it 64 bytes long. Name the program `receive_buffer_LCD_hs`.

Submit your flowchart and list file for this program as part of your prelab.

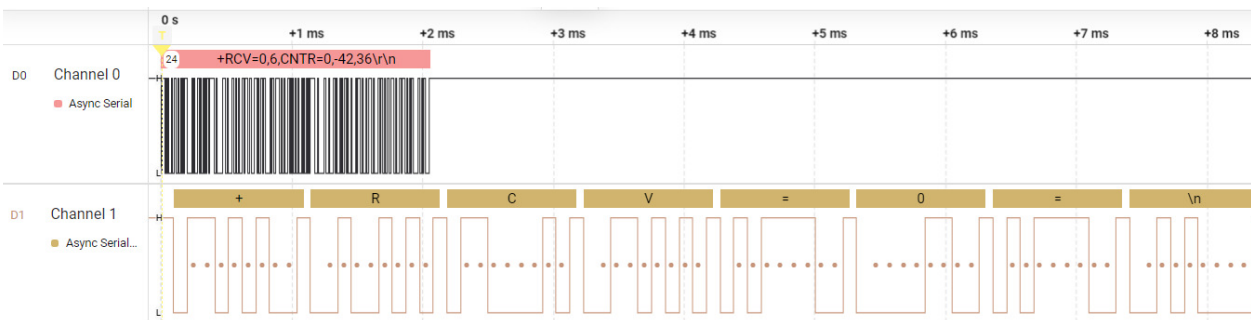
Laboratory Activity

Laboratory Task 1: Receiving RYLR896 Output at 115.2 kbaud and Relaying it to Your LCD at 9600 baud

Use CoolTerm to change the baud rate of your RYLR896 to 115.2 kbaud. Leave its other parameters the same. Use the command AT+IPR=115200 to change the baud rate. After you have changed the baud rate of the RYLR896, you must change the baud rate of CoolTerm to 115.2 kbaud to verify that the RYLR896's baud rate has been successfully changed. Use the AT+IPR? command to do this.

Create a project named `relay_to_LCD_hs` using your program `relay_to_LCD_hs`. Build and debug your program.

Debug here means to make sure your program is doing what you intended it to do. You cannot expect to see the correct information on the LCD. Here is what I got on the Saleae for this task.



Configure the Saleae logic analyzer to monitor both the data received by USART1 and the data transmitted by USART3. Run and debug your program. Use channel 0 for the incoming asynchronous data and channel 1 for the outgoing data. You will need to display the data on two these channels to get the TA's signoff. What you see on channel 1 of the Saleae should be consistent with what you see on the LCD.

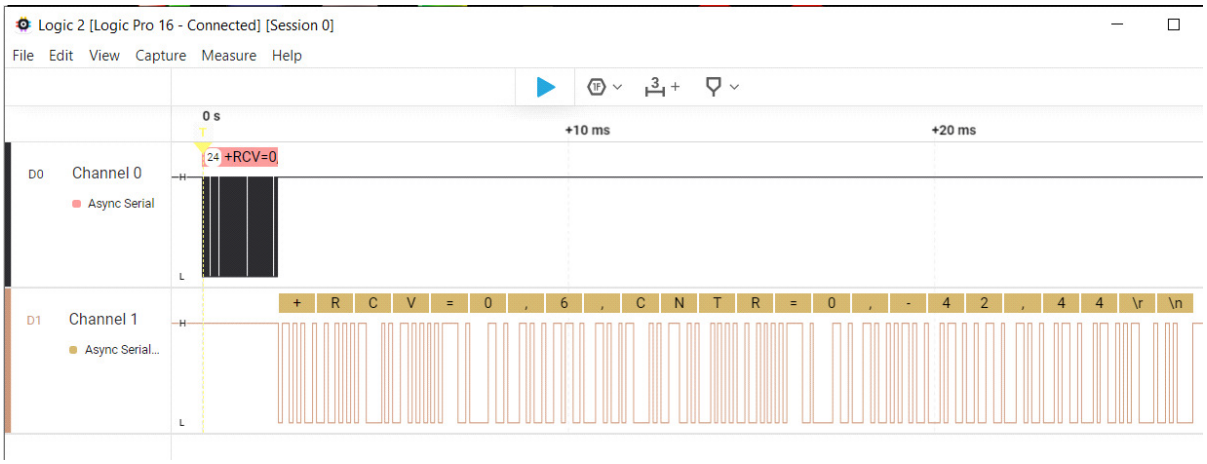
When you have the correct traces on the Saleae Logic Analyzer show them to a TA and get

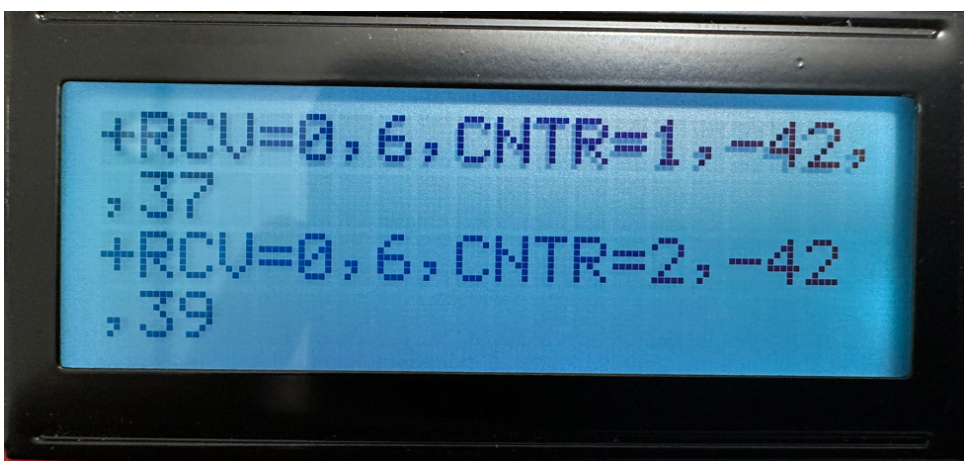
the TA's signature.

Laboratory Task 2: Buffering then Processing the Incoming Data

Create a project named `receive_buffer_LCD_hs` using your program `receive_buffer_LCD_hs`. Build and debug your program.

Configure the Saleae logic analyzer to monitor both the data received by USART1 and the data transmitted by USART3. Run and debug your program. Display the data in the buffer using a watch window and using a memory window set to "dataINTERNAL_SRAM."





Watch 1

Name	Value	Type
BY buffer, 256	0x4000	byte[256]@0x4000
[0]	0x2b	byte@0x4000
[1]	0x52	byte@0x4001
[2]	0x43	byte@0x4002
[3]	0x56	byte@0x4003
[4]	0x3d	byte@0x4004
[5]	0x30	byte@0x4005
[6]	0x2c	byte@0x4006
[7]	0x36	byte@0x4007
[8]	0x2c	byte@0x4008
[9]	0x43	byte@0x4009
[10]	0x4e	byte@0x400a
[11]	0x54	byte@0x400b
[12]	0x52	byte@0x400c
[13]	0x3d	byte@0x400d
[14]	0x33	byte@0x400e
[15]	0x2c	byte@0x400f
[16]	0x2d	byte@0x4010
[17]	0x34	byte@0x4011
[18]	0x32	byte@0x4012
[19]	0x2c	byte@0x4013
[20]	0x33	byte@0x4014
[21]	0x39	byte@0x4015
[22]	0x0d	byte@0x4016
[23]	0x0a	byte@0x4017

Note that my buffer is 256 bytes long.

Memory 4

Memory: data INTERNAL_SRAM Address: 0x4000,data Columns: Auto

```
data 0x4000  2b 52 43 56 3d 30 2c 36 2c 43 4e 54 52 3d 33 2c 2d 34 32 2c 33 39 0d 0a  +RCV=0,6,CNTR=3,-42,39..
data 0x4018  d6 79 1b 5b 9b e0 8d 18 bb e8 20 0d bb 17 ee e9 f4 75 69 d9 c6 f9 ae e7  Öy.[.à..»è .».îéôuiÙÆù®ç
```

When your program is running correctly and your Saleae, watch window, memory window, and LCD are displaying the correct information, demonstrate it to a TA. Get the TA's signature.

Leave the rest of the circuit you have constructed on your breadboard intact, it may be used in later laboratories.